[소프트웨어학과]

[Scored ML-Agent]

[결과보고서]

Version 1.0
2021.12.14

14

학번: 201920792
이름: 유승민
지도교수: 이환용

목 차

요약 4
1 개요 5
1.1 연구 배경 5
1.2 연구 목표 5
2 관련 선행 연구 조사 5
2.1 게임 적용 사례 5
2.1.1 스도쿠 5
2.1.2 바둑 5
2.1.3 F1 2021 5
2.2 알고리즘 6
2.2.1 유저 적응형 강화학습 6
2.2.2 ML + IL 6
3 연구 결과 6
3.1 문제 정의 6
3.1.1 강화학습 6
3.1.2 게임 속 강화학습 6
3.1.3 연구 목표 7
3.2 제안하는 기법 혹은 소프트웨어 구조 7
3.2.1 점수 기법 7
3.2.2 경쟁 기법 8
3.3 학습 세부 사항과 결과 8
3.3.1 score 제한 8
3.3.2 에이전트 꼼수 8
3.3.3 state 제한 9
3.3.4 sc∝st 9
3.3.5 sc∝psc, st∝pst 9
4 성능 분석 10
4.1 성능 분석 환경 10
4.2 성능 분석 결과 11
5 결론 12
참고자료 13

<표 목차>

<그림 목차>

요약

The goal of this project is to control the agent's ability to perform tasks through reinforcement learning by applying a scoring system. Simply put, agents with lower scores will have a lower win rate against agents with higher scores, and agents with higher scores will have higher win rates against agents with lower scores.

For this, the agent should be punished if the state is too good when performing a task in the learning process. At this time, the biggest task is to study these two things: how much punishment should be received and how to establish the correlation between score and state.

※ 결과 보고서의 내용을 영어로 한 두 단락으로 간단하게 작성한다

1 개요

1.1 연구 배경

2021 년 현재 AI 는 활발히 연구되고 있으며 그 영역은 게임업계로도 확장되고 있다. 블레이드&소울의 경우 프로게이머를 뛰어넘는 에이전트를 공개한 적이 있다. 어느 술래잡기 게임에선 에이전트가 인간 이상의 규칙 활용 능력을 보여주며, 심지어 물리엔진의 허점과 버그까지 이용하는 모습도 보여준다.

그러나 앞서 얘기한 완벽한 게임 에이전트들은 실제 게임에 적용되기 어렵다. 에이전트가 너무나 똑똑하여 모든 문제에 최적의 대처 방안을 알고 있는 탓에 게임의 재미를 반감시킬 수 있다. 본 연구는 이러한 문제점을 포착하여 해결법을 제시하기 위해 시작된 연구이며, Unity 와 ML-Agent 플러그인을 이용해 연구를 기획하고 검증한다.

1.2 연구 목표

본 연구의 목표는 게임 에이전트를 원하는 수준의 지능으로 학습시키는 방법론을 만들어내는 것이다. 그를 위해 크게 세 단계의 연구 과정을 거친다. 첫 번째는 방법론을 만드는 것, 두 번째는 방법론을 검증하는 것, 세 번째는 방법론을 적용하는 것이다.

※ 연구의 배경과 목표를 간략하게 기술한다

2 관련 선행 연구 조사

2.1 게임 적용 사례

여기서 다루게 될 사례들은 본 연구의 방향성과는 다르나 본 연구에서 참고하기 좋은 기술을 가지고 있다.

2.1.1 스도쿠

스도쿠 풀이 알고리즘을 가벼운 것부터 순차적으로 적용시켜 시도한 알고리즘 수에 따라 난이도 점수를 매긴다

.

2.1.2 바둑

Rating 시스템을 적용하여 두 경쟁자가 서로 점수를 뺏고 빼앗긴다. 많이 이기는 사람이 많은 점수를 가지게 된다.

2.1.3 F1 2021

AI Score 라는 이름의 점수가 0-110 까지 존재한다. 각 점수는 코스 주행 시간과 매칭되어 AI 의 실력이 정해진다.

2.2 알고리즘

2.2.1 유저 적응형 강화학습

강화학습과 난이도를 키워드로 검색한 결과는 대부분 강화학습으로 게임 난이도를 평가하는 것과 게임 속에서 플레이어에 맞춰 에이전트 난이도가 저장되는 것들이다. 대표적으로 게임을 길찾기 문제로 보고 A*알고리즘으로 게임 난이도를 수치로 표시하는 것과, 이분법적으로 표현하기 힘든 애매한 표현들을 퍼지 이론으로 처리하여 플레이어 실력에 맞춰 게임 난이도가 변화되는 것이 있다.

2.2.2 ML + IL

강화학습과 모방학습을 합쳐 인간 흉내를 내는 강화학습 에이전트를 만들어낸다. 단, 에이전트가 인간처럼 행동할 뿐이지 임무수행능력을 조절하진 못 한다.

※ 본인이 진행하는 연구와 유사하거나 관련이 있는 선행 연구에 대한 조사 결과를 기술한다. ※ 참고 자료에 제시된 본문을 그대로 활용하면 안 되고, 자신의 표현으로 새롭게 재구성하여 작성하여야 하며, 출처를 분명하게 밝혀야 한다 ※ 본 chapter 작성을 위해 참고된 자료들은 모두 Reference 에 제시해야 한다 ※ 중간보고서 대비 차이점은, 연구 결과물이 나왔으므로, 선행 연구와 본인의 연구 결과와의 차별점 등을 기술한다

3 연구 결과

3.1 문제 정의

3.1.1 강화학습

강화학습은 에이전트에게 보상을 제공하며 특정 행동을 강화하도록 유도하는 학습법이다. 이러한 기법의 성질 덕분에 에이전트는 특정 행동을 끝도 없이 강화하여 결국 사람의 수준을 넘어서는 수행 능력을 보이게 된다.

에이전트가 최고의 수행 능력을 가지게 된다는 점은 결점으로 보이는 것이 없어 보인다. 그러나 물론 장점은 곧 단점이 된다. 사람의 수준을 넘어서는 수행 능력은 사람이 에이전트와 경쟁하는 것이 성립하지 않다는 것을 의미한다. 이는 특정 분야에서 골치 아픈 문제로 작용하는데 특히 게임 분야에서 그렇다. 사람이 게임 에이전트와 경쟁하려 하는데 전혀 이길 수 없다면 재미라는 게임의 본질적인 가치가 손상되게 된다.

3.1.2 게임 속 강화학습

게임은 다양한 학습 환경을 쉽게 구축할 수 있다는 장점이 있어 강화학습을 연구하기 좋다. 그러나 아이러니하게도 강화학습을 실제 게임에 집어넣는 것은 어려운 문제다. 너무 뛰어난 에이전트는 사람이 이길 수 없으며 이는 게임의 본질적인 가치인 재미를 손상시킨다. 때문에 게임 업계에선 강화학습은 이벤트성으로만 활용되거나 에이전트의 모션을 학습시키는데 활용하는데 그치며 여전히 실제 에이전트의 경쟁 방식은 학습시키지 않고 개발자가 직접 만들어준다.

3.1.3 연구 목표

본 연구는 3.1.1 과 3.1.2 에서 제시한 문제를 해결하고 강화학습 에이전트가 게임에 활발히 적용될 수 있도록 하는 것을 목표로 하고 있다. 이를 위해 에이전트에게 점수를 매겨 점수와 비례한 임무 수행 능력을 가지도록 하는 기법을 연구한다.

※ **본 연구에서 해결하고자 하는 문제 혹은 연구하고자 하는 내용을 기술한다**
※ **본 연구가 기술적으로 유의미한 이유를 제시한다**
※ **기존의 연구 대비 어떠한 차이점을 갖고 있는지 기술한다**
※ **본 연구의 필요성을 기술한다.**
※ **중간보고서 이후 문제 정의가 수정되었다면, 수정된 이유를 함께 기술한다**

3.2 제안하는 기법 혹은 소프트웨어 구조

3.2.1 점수 기법

본 연구를 위해 에이전트가 가지는 속성들을 S={sc, lsc, psc, st, lst, pst, r}(sc=score of agent, lsc=limit of score, psc purpose of agent, st=state of agent, lst=limit of state, pst=purpose of agent, r=reward value on agent)로 정의한다. Sc∝st, sc∝psc, st∝pst 가 비례함을 가정하고 최종적으로는 이 식이 만족함을 증명함으로써 본 연구를 입증하려 한다. 본 프로젝트 기간 동안에는 결과물로써 st∝pst 를 증명하는데 실패한 사례만 있으며 연구 개념과 다음 연구를 진행하기 위한 유의 사항을 정리하는데 그쳤다.

일반적인 강화학습의 에이전트 (앞으로 최고 에이전트라 부르겠다.)는 임무를 완수했을 때 보상을 받는다. 최고 에이전트는 능력에 한계를 두지 않기에 pst 는 무한대에 수렴한다. 실제로 최고 에이전트는 할 수 있는 것을 모두 동원하여 한계까지 임무 수행 능력을 증가시키며 이는 st∝lst 로 표현할 수 있다. sc∝st 라는 가정하에 lsc∝lst 이며 결국 최고 에이전트는 sc=lsc, st=lst 를 만족해야 한다. 최고 에이전트의 sc,st 는 다른 에이전트가 가질 수 있는 모든 sc, st 보다 크거나 같은 값을 가지며, 따라서 최고 에이전트가 다른 에이전트와 경쟁했을 때의 승률은 50 이하로 내려가지 않아야 한다.

이제 sc 와 st 의 구체적인 수치를 정의한다. 강화학습에는 다양한 학습 환경이 존재하며 각 환경마다 st 의 성질이 다르다. 본 프로젝트에서 사용한 학습 환경에선 에이전트가 스위치를 누르고 피라미드를 파괴하면 보상을 받는다. 이에 피라미드를 파괴한 시간을 st 로 정의하고 따라서 sc 는 피라미드를 파괴한 시간에 비례하게 된다.

St 는 학습 환경에 따라 성질이 변하는 특징을 가지고 있지만, sc 는 학습 환경에 무관하게 성질이 동일하다. 따라서 에이전트의 능력을 묘사할 때 '임무 완수에 0.7sec 이 걸리는' state 를 가졌다고 하지 않고 에이전트의 점수가 500 점이다라고 표현할 수 있다. 이는 강화학습 에이전트 연구 결과가 업계에서 공유되는데 유용하다. 가령 게임을 설계할 때 임무 완수 시간이 0.7sec 인 에이전트의 난이도는 중급이라고 하는 것보다 500 점 에이전트가 중급이라고 통일하는 것이 편하다. 이렇게 되면 다른 게임의 중급 난이도도 500 점 에이전트를 사용하면 된다.

3.2.2 경쟁 기법

3.3 학습 세부 사항과 결과

3.3.1 score 제한

에이전트의 임무 수행 능력을 점수로 제어한다. 이는 에이전트의 능력을 직관적으로 이해하고 결정할 수 있다. 하지만 st 의 성질이 정의되지 않았기에 연구된 에이전트의 sc 가 다른 게임에서 사용되기엔 무리가 있다.

에이전트는 sc 가 psc 에 가까워지면 보상(r=+1)을 받고 멀어지면 처벌(r=-1)을 받는다. 그리고 sc 가 psc 에 가까워지고 있는지 멀어지고 있는지는 sc2-sc1 벡터(sc1: previous score of agent, sc2: now score of agent) 로 표현한다. Sc 는 st 가 좋아지면 증가하고 나빠지면 감소한다. St 가 좋아지고 나빠지는 것은 st2-st1 벡터(st1: previous state of agent, st2: now state of agent) 로 표현한다. 이 경우 Sc∝st 는 성립하지만, 여기서 얻어낸 sc 를 다른 곳에 사용하기엔 무리가 있다. Sc 가 좋아지고 나빠지는 것만으로 보상을 결정하면 다양한 학습 환경에 의한 가변적인 요소를 해결할 수 없다.

최고 에이전트 모델을 가져와 score 제한 기법을 적용하고 학습을 진행했다(학습이 완료된 최고 에이전트를 하락 에이전트라 부르겠다). 하락 에이전트는 임무 완료 시간이 증가했으며 이는 하락 에이전트의 st 값이 낮아졌음을 의미하고 하락 에이전트가 최고 에이전트와 경쟁했을 때 승률은 50 보다 낮을 수밖에 없음을 의미한다.

그러나 많은 문제점이 존재한다. Sc 는 psc 에 수렴하지 않고 계속 상승하며, Sc∝st 는 성립하지만 상관관계를 정의하진 못했고, 에이전트가 꼼수를 부린다. Sc 가 psc 에 수렴하지 않는다면 sc 의 값에 설득력이 떨어진다. Psc가 1500 점이여도 마지막 sc 가 3000 점이라면 너무나 큰 오차 값에 psc 의 값이 의미를 가지기 힘들다. St 와 sc 의 상관관계 정의는 다양한 환경에서 학습을 진행해 어느 sc 의 값이 모든 환경에서 비슷한 st 값을 가져야 비로소 가능하다. 자기주도프로젝트 기간 동안에는 st∝pst 가 성립함을 증명하는데 시간을 모두 사용했으며, Sc∝st 의 성립을 위해서 st 성질이 다른 최소 10 개의 학습 환경에서 학습을 진행해야 한다. 학습환경 구축, st 성질 정의, 학습 진행에 2 개월씩, 그리고 학습결과를 바탕으로 Sc∝st 이 성립함을 증명하는데 2 개월 해서 총 2 년 이상은 걸릴 것으로 예상된다.

3.3.2 에이전트 꼼수

에이전트가 임무 완수를 눈 앞에 두고 가만히 멈춰 서는 꼼수를 부린다. 이는 에이전트가 psc 에 도달하기 위해 sc 를 의도적으로 조정한다고 해석할 수 있다. 이는 게임의 재미요소를 해치기 때문에 프로젝트 목표와는 거리가 멀다.

에이전트가 꼼수를 부리는 원인은 서술논리 분석을 통해 잡아낼 수 있다. 서술논리는 명확히 정의할 수 없는 논리를 명제로써 정의한 논리다. 서술논리 분석을 위해서 가장 먼저 해야 할 것은 에이전트가 언제 보상을 받는지 파악하는 일이며 {GetReward(x, y); x get reward y} 로 표현한다. 3.2.1 기준으로 에이전트는 두 가지 보상을 받는다.

첫 번째로 에이전트가 받는 보상은 각 step 마다 적용되는 GetReward(agent1500, 2f-1f/MaxSteps*ClearSteps)이다. Agent1500 은 psc 가 1500 인 에이전트를 의미하며 2f-1f/MaxSteps*ClearSteps 가 의미하는 것은 에이전트가 임무를 완수한 step 이 느릴수록 보상이 적어진다는 것이다. 즉 x 가 학습하는 것은 임무를 빠르게 수행할수록 y 가 커진다는 것이고 이는 Vx GetLowerClearSteps(x) -> GetBigReward(x)로 표현된다. 다시 해석하자면 모든 x 는 임무 완료 시간을 단축시키면 큰 보상을 받는다는 걸 의미한다.

두 번째로 에이전트가 받는 보상은 sc 가 psc 에 가까워질 때 받는 GetReward(agent1500, 1f)이다. 이는 Vx GetCloseToPSC(x) -> GetReward(x)로 표현되며 다시 해석하자면 모든 x 는 sc가 psc 에 가까워질 때 보상을 받는다는 걸 의미한다.

즉 정리하자면 Vx GetLowerClearSteps(x) V GetCloseToPSC(x) -> GetReward(x)가 된다. 따라서 에이전트는 임무를 빠르게 깨거나, psc 에 가까워지려 하거나 혹은 빠르게 깨면서 psc 에 가까워지기 위해 잠깐 멈추는 꼼수를 부린다. 따라서 GetLowerClearSteps(x)를 제거하면 에이전트는 psc 에 가까워지려고만 하며 꼼수를 부리지 않게 된다.

3.3.3 state 제한

Score 제한은 다양한 학습 환경이 가변적인 요소를 다룰 수 없는 문제점이 있다. 따라서 state 를 제한하고 이를 score 와 매칭하는 과정을 통해 가변적인 요소를 다룬다. 제한하는 방법은 앞과 같으나 몇 가지 개선사항이 있다. 먼저 기존의 sc2-sc1 벡터는 가장 낮은 자리 수까지 계산되기 때문에 아주 미세한 차이로도 psc 에 가까워지는지 멀어지는지가 결정된다. 때문에 매번 우연의 요소로 벡터의 방향이 달라져 변동이 심하다. 이는 에이전트가 psc 에 가까워져야 보상을 받는다는 사실을 학습하는데 혼란을 겪을 수 있다. 따라서 sc2 와 sc1 을 일정 자리 수에서 반올림해 이러한 문제를 해결한다. 이러면 사소한 차이가 있어도 벡터가 0 으로 계산돼 변동을 줄일 수 있다.

3.3.4 sc∝st

sc∝st 의 증명은 다양한 학습 환경을 통해 이루어져야 한다. 동일한 sc 에 대하여 다양한 학습 환경의 에이전트가 보이는 st 가 많은 사람이 보기에 수준이 비슷하다고 느껴야 한다. 이는 더 오랜 시간이 필요하다.

3.3.5 sc∝psc, st∝pst

본 프로젝트에서 진행한 한 학습 환경에 한해서 sc∝st 가 성립한다. 따라서 sc∝psc, st∝pst 중 하나라도 성립하면 나머지도 성립한다. 특히 목표로 한 것은 state 제한을 통한 st∝pst 의 증명이며 이는 pst 를 설정함에 따라 st 가 낮아지는 걸 통해 일차적으로 증명했다. 그러나 st 가 높은 에이전트가 st 가 낮은 에이전트를 상대로 승률이 높게 나오는지를 확인하지 못했고, st 와 sc 가 pst 와 psc 에 수렴하지 않는다는 문제점이 있다.

※ 본 연구에서 문제를 해결하기 위해 제안하는 기법을 기술한다. ※ 기존의 연구 대비 어떠한 차이점을 갖고 있는지 기술한다 ※ 새로운 소프트웨어 구조를 제시하는 경우에는, 제시하는 소프트웨어 구조의 타당성과 구성 요소 각각에 대해 기술한다

4 성능 분석

4.1 성능 분석 환경

본 연구는 Unity 와 ML-Agent 를 이용해 진행했으며 성능 분석으로 Tesorflow Board 를 사용했다.

```
behaviors:
  Pyramids1000:
    trainer_type: ppo
    hyperparameters:
      batch_size: 128
      buffer_size: 2048
      learning_rate: 0.0003
      beta: 0.01
      epsilon: 0.2
      lambd: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
    network_settings:
      normalize: false
      hidden_units: 512
      num_layers: 2
      vis_encode_type: simple
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
      curiosity:
        gamma: 0.99
        strength: 0.02
        network_settings:
          hidden_units: 256
        learning_rate: 0.0003
    keep_checkpoints: 5
    max_steps: 50000000
    time_horizon: 128
    summary_freq: 30000
```

<그림 1> Hyper Parameters

그림 1 은 본 프로젝트에서 모든 학습에서 사용한 Hyper Parameters 를 보이고 있다. 이 Hyper Parameters 를 가지고 Agent 는 미로에서 스위치를 찾아 누르고 그 후 피라미드를 찾아 파괴하면 보상을 받는다는 시나리오를 따라 학습을 진행한다.

본 프로젝트에서는 st 로 정의한 Episode Length 의 최종 값을 이용해 st 가 pst 에 수렴하는지를 판단하고, Episode Length 의 변동을 통해 st 의 변동을 확인한다.

※ 본 연구 결과의 성능 분석을 위한 환경을 기술한다. ※ 시뮬레이션을 수행한 경우, 시뮬레이터와 시뮬레이션의 환경 변수, 시나리오 등을 기술한다 ※ 소프트웨어를 개발한 경우, 적용된 시스템의 OS, CPU, 적용 시나리오 등 소프트웨어의 성능에 영향을 줄 수 있는 요소들이 어떻게 설정되었는지 기술한다 ※ 성능 분석 metric 에 어떠한 것들이 있는지, 그 의미가 무엇인지, 그리고 어떻게 측정할 것인지 기술한다

4.2 성능 분석 결과

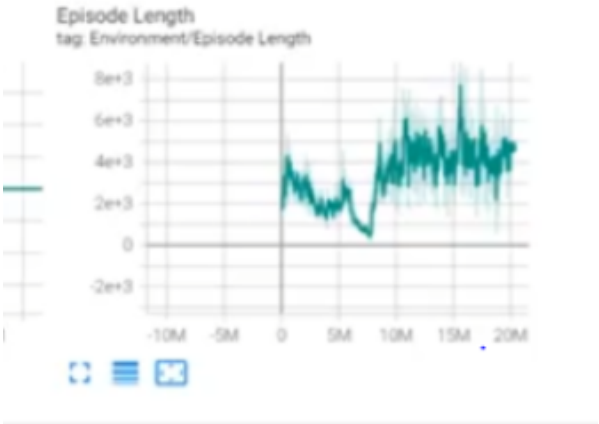
※ 4.1 에 제시된 성능 분석 환경에서 성능을 분석한 결과를 그래프 혹은 표를 사용하여 제시한다 ※ 그래프와 표에 주어진 수치 혹은 변화 경향이 의미하는 바가 무엇인지 해석한다

<그림 2> 최고 에이전트의 st

그림 2 는 최고 에이전트의 st(Episode Length) 그래프를 보여준다. St 가 낮을수록 좋은 것이며 최고 에이전트 답게 st 의 값이 무척 낮은 곳에 수렴하고 있다.

<그림 3> 최고 에이전트 점수 기법 결과

그림 3 은 그림 2 보다 st 가 높은 곳에 수렴하고 있다. 점수 기법의 목표가 다소 이뤄줬으나 에이전트가 꼼수를 부린다는 문제점이 발생했다.



<그림 4> 일반 에이전트 점수 기법결과

그림 4 는 점수 기법을 적용해 낮은 임무 수행 능력을 가지는 에이전트를 학습시켰을 때의 st 그래프를 보인다. 에이전트의 st 길이가 무척 길어졌다. 다만, st 의 변동이 너무 크다는 문제점이 있다.

5 결론

정리하자면 본 프로젝트에서 이뤄낸 것은 연구 내용을 정의하고 실패한 사례를 다룬 것뿐이다. 연구 목표인 에이전트에게 점수를 매겨 점수에 비례하는 임무 수행 능력을 가지도록 하는 것은 예측할 수 없는 변수가 너무 많이 발생하기 때문에 굉장히 어려운 주제이다. 학습에 대략 8 시간이 걸리는데 코드 한 줄만 잘못 짜여져 있기라도 하면 8 시간이 의미 없는 시간이 된다. Sc∝st, sc∝psc, st∝pst 를 증명함으로써 본 연구가 완료된다는 것만 해도 정의하기까지 몇 개월이 걸렸다.

연구 성과는 목표의 33%로도 달성하지 못했다. Sc∝st, sc∝psc, st∝pst 세 가지 중 sc∝pst 조차 정의하고 입증하는데 완벽하지 못했다. 그러나, 본 연구를 완료하기 위해 필요한 것들을 정의하고 다양한 실패 사례를 통해 얻어낸 유의점들은 다음 연구의 큰 밑거름이 될 수 있다.

기존의 연구는 대부분 유저 맞춤형 강화학습 에이전트를 연구하고 있다. 이는 에이전트의 난이도가 가변적이고 정립되지 않는다는 문제점이 있다. 가령, 유저 맞춤형 에이전트를 상태로 보통 난이도를 깼다고 하는 것은 더 이상 유용한 단어가 아니다. 본 연구는 점수를 통해 에이전트의 능력을 정의한다는 차별점이 있고 이는 게임 업계에서 난이도를 설정하는 새로운 규칙으로 자리 잡을 가능성이 있다.

본 프로젝트를 발전시켜 연구 목표를 달성시킬 수만 있다면 Agent 를 개발하는데 소요되는 시간, 자금을 대폭 줄일 수 있다. 또한 기존의 Agent 와는 다른 혁신적인 Agent 로써 사용되어 새로운 패러다임을 불러일으킬 수 있다.

※ 연구 목표를 다시 한 번 제시하고, 이 목표를 달성하기 위해 어떤 기법 혹은 소프트웨어를 개발하였는지 기술한다 ※ 연구 성과가 목표 대비 어느 정도 만족하고 있는지 기술한다 ※ 기존의 연구들과 비교하여 본 연구 성과가 어떤 차별점 혹은 중요한 의미를 갖는지 기술한다 ※ 본 연구를 발전시켜 나간다고 할 때 어떤 연구가 가능할지 제시한다

참고자료

[1]. "강화학습을 이용하여 프로게이머 수준의 <Blade and Soul> 비무 AI 만들기", NDC REPLAY, 2019 년, http://ndcreplay.nexon.com/NDC2019/sessions/NDC2019_0061.html

[2]. "Go ranks and ratings", Wikipedia, https://en.wikipedia.org/wiki/Go_ranks_and_ratings

[3]. "Sudoku Difficulty Ratings", Sudoku, https://www.sudokuoftheday.com/about/difficulty/

[4]. "F1 2021 Game: How to get the right AI level every time", RACINGGAMS.GG, 2021.08.17, https://racinggames.gg/f1/f1-2021-game-how-to-get-the-right-ai-level-every-time-career-mode-my-team-difficulty/

[5] 진영재, 오경수, "Difficulty Evaluation of Game Levels using A Path-Finding Algorithm", 한국게임학회 논문지 = Journal of Korea Game Society v.15 no.4 , 2015 년, pp.157 - 168

[6] 박창훈, 서진석, "퍼지이론을 이용한 슈팅게임 난이도 조절", 멀티미디어학회논문지 = Journal of Korea Multimedia Society v.20 no.8 , 2017 년, pp.1464 - 1471

[7] Julien Dossa, Rousslan ,Fernand , "A Human-Like Agent Based on a Hybrid of Reinforcement and Imitation Learning", https://library.ajou.ac.kr/#/eds/detail?an=edseee.8852026&dbId=edseee, 2019 년

※ 본 보고서 작성에서 참고되고 인용된 자료들을 모두 기술한다 ※ 논문, 보고서, 책, 웹 사이트, 블로그 등 모든 형태의 참고 자료를 기술한다 ※ 참고 자료 기술 형식은 IEEE reference 작성 기준에 따라 작성한다.